Lecture 12: UDP Client-Server Programming II

Objectives:

- Learn the good points about UDP applications
- Learn about problems associated with UDP applications and how to handle them.
- Learn how to control the behavior of Socket using Socket Options
- Learn how to catch SocketException and find the cause of the error

1. Good Points about UDP Applications

1.1 Message Boundary:

Unlike TCP, UDP preserves message boundaries between clients and servers.

There are no internals buffers. Thus, when a UDP packet is sent from a source using one call to the ***SendTo*** method, the packet is directly forwarded to the recipient, and the recipient will receive it using one call to the ***ReceiveFrom*** method.

This 1-1 correspondence makes programming a UDP client-server applications easier than TCP where message boundary is not respected.

For example, the following shows a server sending messages by calling the ***SendTo*** method 5 times and a client receiving then using 5 calls to the ***ReceiveFrom*** method:

Server:

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

class UdpBoudaryTestServer
{
   public static void Main()
   {
      IPEndPoint localEP = new IPEndPoint(IPAddress.Any, 9050);
      Socket server = new Socket(AddressFamily.InterNetwork,
                      SocketType.Dgram, ProtocolType.Udp);
      server.Bind(localEP);
      Console.WriteLine("Waiting for a client...");
```

```
        //dummy end-point
        EndPoint remoteEP = new IPEndPoint(IPAddress.Any, 0);

        byte[] data;
        int recv;

        for(int i = 0; i < 5; i++)
        {
            data = new byte[1024];
            recv = server.ReceiveFrom(data, ref remoteEP);
            Console.WriteLine("Received message from {0}: {1}", remoteEP,
                              Encoding.ASCII.GetString(data, 0, recv));
        }
        server.Close();
    }
}
```

Client:

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

class TestUdpClient
{
    public static void Main()
    {
        IPEndPoint remoteEP = new IPEndPoint(
                        IPAddress.Parse("127.0.0.1"), 9050);

        Socket client = new Socket(AddressFamily.InterNetwork,
                        SocketType.Dgram, ProtocolType.Udp);

        for (int i=0; i<5; i++)
            client.SendTo(Encoding.ASCII.GetBytes("Test Message "+i),
remoteEP);

        Console.WriteLine("Messages sent");
        Console.WriteLine("Stopping client");
        client.Close();
    }
}
```

1.2 Speed:

Because there are no connection/disconnection overheads and because there is no buffering and other error checks, UDP packets gets to their destination (if they do) much faster.

2. Some problems associated with UDP

2.1 Communicating Partner not responding:

In the example above, the server makes five calls to the **ReceiveFrom** method in a loop. The problem is, what if the client sends less than five messages? Also what if some of the messages get lost before they reach the server?

Since the **ReceiveFrom** method is a blocking method, the server will be waiting indefinitely for the next packet.

A solution to this problem is to use **socket option** to set a receive time-out for the socket.

The following section explains how to set socket options.

2.1.1 Socket Options

The Socket class provides the SetSocketOption method that can be used to set protocol options for the created Socket object.

The method is overloaded, using three different formats as follows:

```
SetSocketOption(SocketOptionLevel sl, SocketOptionName sn, byte[]
value)
SetSocketOption(SocketOptionLevel sl, SocketOptionName sn, int
value)
SetSocketOption(SocketOptionLevel sl, SocketOptionName sn, object
value)
```

The method allows different option levels to be modified namely, the socket itself or the underlying protocols (UDP, TCP or IP).

Accordingly, the parameter *sl* can take one of the values of the SocketOptionLevel enumeration listed below:

Value	Description
IP	Options for IP sockets

Value	Description
Socket	Options for the socket
Tcp	Options for TCP sockets
Udp	Options for UDP sockets

The *sn* specifies the name of the specific socket option that is being set.

The following table shows some of the SocketOptionNames and their SocketOptionLevel. For a full list of possible values, refer to the documentation or chapter 3 of the reference book.

Value	**SocketOptionLevel**	**Description**
AddMembership	IP	Adds an IP group membership
DropMembership	IP	Drops an IP group membership
Broadcast	Socket	If true, permits sending broadcast messages
DontLinger	Socket	Closes socket gracefully without waiting for data
Linger	Socket	Waits after closing the socket for any extra data
IpTimeToLive	IP	Sets the IP packet time-to-live value
KeepAlive	Socket	Sends TCP keep-alive packets
MulticastTimeToLive	IP	Sets the IP multicast time to live
NoChecksum	Udp	Sends UDP packets with checksum set to zero
ReceiveBuffer	Socket	Sets the total per-socket buffer reserved for receiving packets
ReceiveTimeout	Socket	Sets the Receives time-out value
SendBuffer	Socket	Sets the total per-socket buffer reserved for sending packets

Value	SocketOptionLevel	Description
SendTimeout	Socket	Sets Sends time-out value

The *value* parameter defines the value of the socket option name to set. The format of the value is different depending on the SocketOptionName used.

Note that for socket options that require boolean values such as **Linger**, an integer value is used with 0 for false and 1 for true.

The Socket class also has **GetSocketOption** method that can be used to get the current settings for a socket option.

Again, the method is overloaded as shown below:

```
object GetSocketOption(SocketOptionLevel sl,  SocketOptionName sn)
void GetSocketOption(SocketOptionLevel sl, SocketOptionName sn,
byte[] value)
```

For options that have int or object values, the first version is used, while for options that have byte[] values, the second version is used.

2.1.2 Solving the "Partner not responding problem"

To solve the problem of ReceiveFrom blocking indefinitely, we set the **ReceiveTimeOut** option of the socket.

The value for ReceiveTimeOut is an integer representing the time to wait in mili seconds.

The following code sets the server socket to give up after waiting for 10 seconds.

```
server.SetSocketOption(SocketOptionLevel.Socket,

SocketOptionName.ReceiveTimeout, 10000);
```

Note that after the 10 seconds, a SocketException is thrown, thus the calls to the ReceiveFrom method should be done inside try block.

2.2 Lost data due to small array size or network error:

The ***ReceiveFrom*** and the ***SentTo*** methods of the Socket class use a byte array to receive and send data.

If the array is too small to receive the packet, since there is no buffering, part of the data will be lost. In such a case, a SocketException will be thrown.

A solution is to catch the SocketException, increase the array size and request the sender to re-transmit.

The SocketException has a field, **ErrorCode** which returns a numeric value corresponding to the actual error condition that caused the SocketException to be thrown.

For a complete list of error codes, refer to Chapter 6, page 243.

Based on our discussion in this section, the following two error codes are relevant:

10040	Message too long – when the receiving data array is too small
10054	Connection reset by peer – when ReceiveMethod times out.

The following example shows how to use the **ReceiveTimeout** option and how to use the **SocketException** to take care of too small data buffer problem.

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

class BestUdpClient
{
   private static byte[] data = new byte[1024];
   private static int size = 10;


 private static int AdvancedSendReceive(Socket s, byte[] message,

                              EndPoint remoteEP)
   {
      int recv = 0;
      int retry = 0;

      while (true)
      {
         Console.WriteLine("Attempt #{0}", retry);
         try
```

```
            {
                s.SendTo(message, message.Length, SocketFlags.None, remoteEP);
                data = new byte[size];
                recv = s.ReceiveFrom(data, ref remoteEP);
             return recv;
            } catch (SocketException e)
            {
                if (e.ErrorCode == 10054) //receive timeout
                    recv = 0;
                else if (e.ErrorCode == 10040) //buffer too small
                {
                    Console.WriteLine("Error receiving packet");
                    size += 10;
                    recv = 0;
                }
            }

            if (recv == 0)
            {
                retry++;
                if (retry > 4)
                {
                    return 0;
                }
            }
        }
    }

    public static void Main()
    {
        IPEndPoint remoteEP = new IPEndPoint(IPAddress.Parse("127.0.0.1"),

             9050);

        Socket client = new Socket(AddressFamily.InterNetwork,
                        SocketType.Dgram, ProtocolType.Udp);

        client.SetSocketOption(SocketOptionLevel.Socket,

SocketOptionName.ReceiveTimeout, 5000);

        string input, echo;
        int recv;

        while(true)
        {
            Console.Write("Enter message for the server: ");
            input = Console.ReadLine();
            if (input == "exit")
                break;
            recv = AdvancedSendReceive(client,
                                                Encoding.ASCII.GetBytes(input),
remoteEP);
```

```
            if (recv > 0)
            {
                echo = Encoding.ASCII.GetString(data, 0, recv);
                Console.WriteLine("Received from server: " + echo );
            } else
                Console.WriteLine("Did not receive an answer");
        }
        Console.WriteLine("Stopping client");
        client.Close();
    }
}
```

Note, you can test the above client using the **UdpSocketServer**.

3. Do not convert UDP to TCP

From the above discussion, it is obvious that many things can go wrong with UDP transmission.

Unfortunately, UDP does not have any internal support for handling such problems.

As we saw in the last example, one can programmatically implement some sort of error checking mechanism to solve some of these problems.

However, overdoing this will clearly defeat the purpose of UDP, as any such mechanism will require re-transmission and other overheads which could slow-down the UDP transmission.

Therefore as a rule, if your application requires reliability, just go for TCP instead of UDP.